CapsuleReload: HMR + Agent 기반 런타임 게임 코드 생성 및 적용

초안 버전: 소개용 + 논문화 전략 포함

---

1. 한 줄 소개

CapsuleReload는 실행 중인 게임의 특정 gameplay behavior를 에이전트가 생성·수정하고, 상태를 보존한 채 안전하게 적용하며, 실패 시 롤백까지 수행하는 런타임 코드 갱신 프레임워크를 지향하는 아이디어다.

조금 더 쉽게 말하면, 이 프로젝트는 다음 질문에서 출발한다.

> **"게임을 껐다 켜지 않고, 플레이 상태를 유지한 채, 새로운 규칙이나 행동을 즉시 삽입할 수 없을까?"**

그리고 그 답을 단순 hot reload가 아니라
에이전트 기반 코드 생성 + 상태 보존형 적용 프로토콜로 만들려 한다.

---

2. 왜 필요한가

게임 개발에서 가장 비싼 비용 중 하나는 반복 루프다.

- 아이디어를 수정한다.
- 코드를 고친다.
- 다시 빌드한다.
- 다시 실행한다.
- 이전 플레이 상태를 다시 재현한다.
- 결과를 확인한다.
- 만족스럽지 않으면 다시 반복한다.

이 루프는 개발 단계에서도 비싸고, 실험적 게임 디자인에서는 더 치명적이다.
특히 다음 상황에서 비효율이 크다.

- 작은 gameplay rule을 자주 바꿔야 하는 경우
- NPC behavior를 빠르게 실험해야 하는 경우
- 플레이 중 나타난 상황에 맞게 동적으로 행동을 추가하고 싶은 경우
- 생성형 시스템이 새 규칙을 만들 수는 있지만, 이를 실제 런타임에 안전하게 적용하기 어려운 경우

즉, 문제는 "코드를 생성할 수 있느냐"가 아니라
"실행 중인 게임에 그것을 안전하게 적용할 수 있느냐"에 있다.

3. 기존 접근의 한계

3.1 기존 hot reload / HMR의 한계

전통적인 hot reload나 HMR은 빠른 개발 피드백에는 매우 유용하다.
하지만 대체로 다음을 전제한다.

- 변경 코드는 사람이 직접 작성한다.
- 변경 범위가 비교적 단순하다.
- 상태 마이그레이션은 제한적이거나 수동이다.
- 잘못 적용되면 개발자가 직접 복구한다.

즉, 기존 HMR은 **사람이 통제하는 수정 루프**에 잘 맞는다.
하지만 "에이전트가 새 로직을 만들고, 그것을 실행 중인 게임에 넣는다"는 문제는 별개다.

3.2 생성형 게임 시스템의 한계

최근의 생성형 시스템은 다음을 할 수 있다.

- 장면 생성
- 오브젝트 배치
- 코드 스니펫 생성
- 룰 또는 behavior 초안 작성
- 자연어로 게임 변경 요청 처리

하지만 이런 시스템은 대체로 아래 중 하나에 머무른다.

- 개발 시간(dev-time) 생성
- 에디터 보조
- 프로토타입 수준의 장면 구성
- 전체 재시작을 전제로 한 수정

즉, "실행 중인 게임의 상태를 유지한 채 새로운 behavior를 넣는 문제"는 여전히 어렵다.

3.3 핵심 공백

이 분야의 진짜 공백은 다음 세 가지다.

1. **상태 보존**
    - 코드를 바꾸면 기존 객체 상태, 퀘스트 상태, 인벤토리, AI 내부 상태를 어떻게 유지할 것인가
2. **안전한 적용**
    - 생성된 코드가 잘못되었을 때 게임을 망가뜨리지 않으려면 어떻게 할 것인가

3. **실시간성**
    - 적용 지연과 프레임 멈춤을 얼마나 줄일 수 있는가

따라서 문제는 단순 code generation이 아니라
state-preserving runtime behavior replacement다.

---

4. 핵심 아이디어

CapsuleReload의 핵심 아이디어는 다음과 같다.

> **게임 전체를 hot reload하는 것이 아니라, gameplay behavior를 교체 가능한 capsule 단위로 캡슐화하고, 에이전트는 이 capsule에 대해서만 코드를 생성·수정하며, 런타임은 명시적 update point와 state migration, guarded apply, rollback으로 이를 안전하게 적용한다.**

즉, 이 프로젝트는 세 가지를 합친다.

- HMR / hot reload의 빠른 적용성
- 에이전트 기반 코드 생성의 유연성
- 동적 소프트웨어 업데이트(DSU) 계열의 상태 전이 사고방식

---

5. 이 프로젝트가 실제로 하려는 일

이 아이디어를 구성 요소로 나누면 다음과 같다.

5.1 Gameplay Capsule

게임의 모든 코드를 수정 대상으로 두지 않는다.
대신 다음처럼 **교체 가능한 behavior 단위**를 정의한다.

- NPC decision policy
- 무기 효과 규칙
- 퀘스트 트리거 로직
- 퍼즐 상호작용 규칙
- 적 스폰 규칙
- 보상 계산 로직
- 이벤트 반응 정책

이 캡슐은 다음 성질을 가져야 한다.

- 외부와의 인터페이스가 비교적 명확할 것
- 상태를 구조적으로 분리할 수 있을 것
- 프레임 경계나 특정 이벤트 경계에서 교체 가능할 것
- 실패 시 이전 버전으로 롤백 가능할 것

핵심은, **무엇이 hot-swappable한지 먼저 정의하는 것**이다.

5.2 Explicit Update Point

아무 때나 코드를 갈아끼우면 위험하다.
따라서 업데이트는 반드시 명시적 시점에서만 일어나야 한다.

예시:

- 프레임 종료 시점
- 특정 ECS phase 경계
- AI tick 경계
- 이벤트 처리 완료 직후
- 대화 세션 종료 시점

이 시점에서는 최소한 다음이 보장되어야 한다.

- 현재 capsule이 중간 실행 상태가 아닐 것
- 상태 스냅샷이 가능할 것
- 새 버전 적용 전 검증을 거칠 수 있을 것

5.3 State Schema and Migration

진짜 어려운 부분은 코드가 아니라 상태다.
기존 behavior에서 새 behavior로 넘어갈 때 상태를 어떻게 넘길 것인가가 핵심이다.

예를 들어 NPC merchant behavior를 바꾼다고 하자.

기존 상태:

- inventory
- relationship score
- active quest flags
- discount cooldown
- current dialogue branch

새 behavior가 추가 정보를 필요로 하면 상태 migration이 필요하다.

예시:

- `discount_strategy_version`
- `player_reputation_bucket`
- `last_offer_history`

따라서 capsule에는 단순 코드뿐 아니라 다음이 포함되어야 한다.

- state schema version
- migration function

- compatibility rule
- default initialization rule

즉, runtime replacement는 코드 교체가 아니라
code + state transition 문제로 다뤄야 한다.

5.4 Guarded Apply

에이전트가 만든 코드는 바로 적용되면 안 된다.
적용 전에는 최소한 다음 검사를 거쳐야 한다.

- 문법 및 타입 검사
- 컴파일 가능성
- interface compatibility
- capsule contract 준수 여부
- sandbox simulation 또는 dry run
- state migration validation
- runtime guard condition 확인

이 모든 검사를 통과한 뒤에만 실제 게임에 적용한다.

5.5 Rollback

실패 가능성은 전제로 둬야 한다.
따라서 좋은 시스템은 "절대 실패하지 않는다"가 아니라
"실패해도 안전하게 돌아간다"를 목표로 해야 한다.

Rollback은 최소한 아래를 만족해야 한다.

- 이전 capsule binary/script를 보존
- 이전 state snapshot을 보존
- 적용 실패 시 원자적으로 복귀
- 복귀 원인과 실패 로그 기록
- 이후 에이전트 재시도 가능

5.6 Tool-first Edit + Selective Codegen

이 프로젝트는 모든 변경을 raw code generation으로 처리하지 않아도 된다.
오히려 실용적인 구조는 다음과 같다.

- 구조적이고 반복 가능한 변경은 tool call로 수행
- 새 규칙이나 새 behavior logic만 제한적으로 codegen
- 생성 범위를 capsule 내부로 제한
- 외부 엔진 레벨 코드는 건드리지 않음

즉, "에이전트가 게임 전체를 다시 쓰는" 것이 아니라
런타임이 허용한 좁은 영역 안에서 behavior patch를 만드는 것이 더 안전하고 논문으로도 강하다.

---

6. 예시 시나리오

다음은 이 아이디어를 설명하기 좋은 대표 시나리오다.

시나리오: 상인 NPC 행동 수정

상황:

- 플레이어가 특정 퀘스트를 완료했다.
- 디자이너는 즉석에서 "이제부터 상인 NPC는 평판이 높은 플레이어에게 묶음 할인과 대체 대사를 제공했으면 좋겠다"고 요청한다.

일반적인 방법:

- 코드를 수정
- 빌드
- 게임 재실행
- 퀘스트 상태를 다시 재현
- 결과 확인

CapsuleReload 방식:

1. 에이전트가 상인 NPC behavior capsule을 식별
2. 새 할인 규칙과 대화 분기 로직을 생성
3. 필요한 새 상태 필드를 정의
4. migration function을 생성
5. sandbox에서 contract 체크
6. 다음 update point에서 capsule을 교체
7. 기존 인벤토리, 퀘스트 상태, 관계 점수는 유지
8. 실패 시 이전 behavior로 자동 복귀

이 시나리오는 이 프로젝트가 단순 hot reload가 아니라
상태를 유지하며 behavior를 바꾸는 시스템이라는 점을 잘 보여준다.

---

7. 무엇이 새로울 수 있는가

이 프로젝트의 새로움은 "hot reload도 있다, LLM도 있다" 자체에 있지 않다.
진짜 새로움은 다음 조합에 있다.

7.1 Behavior Boundary의 명시적 정의

무엇이 교체 가능한 단위인지 명시한다.

7.2 State-preserving Runtime Update

코드와 상태를 함께 다룬다.

7.3 Agent-generated Patch with Guardrails

에이전트가 패치를 만들지만, 허용된 경계와 검증 절차 안에서만 적용한다.

7.4 Rollback-aware Live Authoring

실행 중 수정의 실패를 정상 상태로 다루고, 이를 시스템 설계의 일부로 넣는다.

7.5 Runtime Gameplay Authoring

에디터 수준 생성이 아니라, 플레이 상태를 가진 런타임 게임에 직접 적용한다.

즉, 이 프로젝트는
hot reload를 AI 시대의 runtime software update 문제로 다시 정의하는 시도라고 볼 수 있다.

---

8. 무엇을 주장하지 않을 것인가

이 아이디어를 소개할 때는 아래를 과장하면 안 된다.

- "게임 전체를 마음대로 실시간으로 바꿀 수 있다"
- "에이전트가 엔진 내부까지 완전히 자동 수정한다"
- "어떤 코드든 안전하게 hot patch가 가능하다"
- "모든 장르와 엔진에 바로 일반화된다"

더 정확한 표현은 다음과 같다.

- 이 시스템은 **사전에 정의된 gameplay capsule 경계 안에서 동작한다.**
- 목표는 **범용 자기수정 코드**가 아니라 **제한된 behavior patching**이다.
- 핵심은 **상태 보존과 안전한 적용 프로토콜**이다.
- 일반성보다는 **명확한 경계와 측정 가능한 효과**를 중시한다.

---

9. 왜 이 프로젝트가 논문이 될 수 있는가

이 프로젝트는 언뜻 보면 "재미있는 시스템 아이디어"처럼 보일 수 있다.
하지만 논문은 다음처럼 잡으면 충분히 성립한다.

핵심 질문:

> **실행 중인 게임에서, 에이전트가 생성한 behavior patch를 상태를 보존한 채 안전하게 적용할 수 있는가?**

이 질문은 단순 구현 문제가 아니라, 다음을 함께 요구한다.

- 교체 가능한 behavior 경계 정의
- 상태 migration 모델
- 안전한 apply protocol
- rollback 설계
- latency와 pause 측정
- failure mode taxonomy

즉, 이것은 단순 데모가 아니라
런타임 게임 행동 생성과 적용을 위한 시스템 설계 및 평가 문제다.

---

10. 논문 H1: State-Preserving Agentic Hot Reload for Runtime Gameplay Behaviors

10.1 문제 정의

현재의 게임 생성 또는 hot reload 시스템은 다음 중 하나에 치우치는 경우가 많다.

- 생성은 되지만 실행 중 안전한 적용이 어렵다.
- reload는 되지만 상태 보존과 migration이 빈약하다.
- 데모는 가능하지만 latency, rollback, failure handling이 연구적으로 정리되지 않았다.

즉, 핵심 문제는
runtime behavior generation이 실제 live authoring 수준으로는 아직 불안정하다는 점이다.

10.2 핵심 주장

> **Gameplay capsule, explicit update point, state migration, guarded apply, rollback을 결합하면, 실행 중인 게임에서 agent-generated behavior patch를 더 낮은 pause와 더 높은 state preservation correctness로 적용할 수 있다.**

10.3 이 논문의 핵심 기여

- gameplay capsule이라는 교체 단위 개념 제안
- code + state migration을 함께 다루는 runtime patch protocol 제시
- guarded apply 및 rollback 설계
- runtime gameplay behavior patching을 위한 평가 항목 정리
- 실제 프로토타입 구현 및 사례 기반 평가

10.4 시스템 범위

이 논문은 범위를 좁게 잡는 것이 중요하다.

포함:

- script-level 또는 component-level gameplay behaviors
- 명시적 인터페이스를 가진 capsule
- 제한된 state migration
- sandboxed patch generation과 적용

제외:

- 엔진 코어 전체 교체
- arbitrary native memory patching
- 완전 자유형 self-modifying code
- 모든 상태의 완전 자동 migration

10.5 평가 설계

대표 태스크

- NPC behavior 변경
- 무기 효과 규칙 교체
- 퍼즐 로직 수정
- 퀘스트 조건 추가
- 이벤트 반응 규칙 변경

비교 대상

- 전체 재시작 + 수동 수정
- 일반 hot reload without migration
- agentic generation without guarded apply
- full CapsuleReload system

핵심 지표

- End-to-end patch latency
- Runtime apply pause time
- Compile/apply success rate
- State preservation correctness
- Rollback success rate
- Visible failure rate
- Human intervention count

추가로 볼 수 있는 지표

- Designer task completion time

- perceived liveness
- number of retries
- patch locality
- failure cause distribution

10.6 예상 목차

1. Introduction
2. Background and Motivation
3. Why Conventional Hot Reload Is Not Enough
4. Gameplay Capsule Model
5. Runtime Patch Protocol
6. State Migration and Rollback
7. Prototype Implementation
8. Experimental Evaluation
9. Failure Analysis and Limitations
10. Related Work
11. Conclusion

10.7 왜 논문이 되는가

이 논문의 공헌은 "에이전트로 게임 코드를 생성했다"가 아니다.

더 정확히는 다음과 같다.

> **실행 중 게임 코드 생성의 핵심 문제를 '어떻게 생성할 것인가'가 아니라 '어떻게 상태를 보존하며 안전하게 적용할 것인가'로 재정의하고, 이를 위한 시스템 구조를 제안하고 평가한다.**

즉, 새로움은 코드 생성 그 자체보다
runtime application protocol과 state model에 있다.

10.8 어떤 평가가 들어가야 설득력이 생기는가

이 논문은 단순 데모 영상으로는 약하다.
반드시 다음이 필요하다.

- 정량 지표
- baseline 비교
- 실패 사례 분석
- rollback이 실제로 동작하는 예시
- state migration 성공/실패 사례
- 적용 가능한 capsule 범위 설명

즉, "멋진 데모"가 아니라
언제 되고, 언제 안 되며, 무엇 때문에 실패하는지를 써야 논문이 된다.

---

11. 이 프로젝트의 현실적 리스크

이 아이디어는 분명 매력적이지만, 리스크도 크다.

11.1 구현 난이도

상태 migration과 rollback은 생각보다 어렵다.

11.2 엔진 종속성

Unity, Godot, Unreal, custom engine 중 어느 것을 선택하느냐에 따라 구현 방식이 크게 달라진다.

11.3 평가 설계 난이도

"좋아졌다"를 무엇으로 측정할지 설계가 어렵다.

11.4 시스템 범위 통제

범위를 너무 넓게 잡으면 구현도, 논문도 둘 다 약해진다.

따라서 이 프로젝트는 grand vision으로 출발하기보다
작은 capsule set과 제한된 task suite로 시작하는 것이 맞다.

---

12. 개발 전략

이 아이디어를 실제 프로젝트로 밀고 간다면 다음 순서가 좋다.

1단계: 작은 prototype

- 하나의 엔진 선택
- 2~3개의 capsule 유형만 정의
- update point 설계
- 최소한의 migration/rollback 구현

2단계: task suite 구성

- NPC behavior
- quest trigger
- weapon modifier 정도만으로도 충분하다.

3단계: baseline 확보

- full restart

- naive hot reload
- guarded apply 없는 version

4단계: 논문화

- 시스템 논문 1편으로 먼저 정리
- 필요하면 이후 benchmark나 HCI 확장 논문으로 분화

현재로서는 **논문 1편으로 수렴시키는 전략이 가장 좋다.**

---

13. 이 아이디어를 소개할 때의 추천 문장

짧은 소개

CapsuleReload는 실행 중인 게임에서 특정 gameplay behavior를 agent-generated patch로 교체하되, 상태를 보존하고 실패 시 롤백 가능한 runtime hot reload framework를 지향하는 연구 아이디어다.

조금 긴 소개

이 프로젝트는 단순 hot reload나 코드 생성 데모가 아니라, gameplay behavior를 교체 가능한 capsule 단위로 정의하고, 에이전트가 생성한 패치를 explicit update point, state migration, guarded apply, rollback 프로토콜을 통해 실제 실행 중인 게임에 안전하게 적용하는 방법을 탐구한다.

연구 소개용 표현

CapsuleReload의 연구적 핵심은 "게임 코드 생성"보다 "생성된 behavior를 상태를 유지하며 어떻게 안전하게 적용할 것인가"에 있다.

---

14. 최종 요약

CapsuleReload는 다음 문장으로 요약할 수 있다.

> **CapsuleReload는 에이전트가 생성한 gameplay patch를 실행 중인 게임에 상태 보존형으로 적용하기 위한 runtime update framework다.**

그리고 논문화는 다음 한 편으로 가는 것이 가장 자연스럽다.

- **논문 H1:** State-Preserving Agentic Hot Reload for Runtime Gameplay Behaviors

이 논문에서 중요한 것은 "얼마나 화려한 코드를 생성했는가"가 아니라,

- 어떤 경계를 hot-swappable하게 정의했는가
- 상태를 어떻게 넘겼는가
- 적용 지연을 얼마나 줄였는가

- 실패 시 어떻게 복구했는가
- 실제로 live authoring에 가까운 경험을 제공하는가

를 보여주는 것이다.

즉, 이 프로젝트는 단순한 게임 데모가 아니라
AI 시대의 런타임 소프트웨어 업데이트 문제를 게임이라는 가장 까다로운 환경에서 다루는 연구 아이디어로 정리할 수 있다.